

[51] Learned Cardinality Estimation for Similarity Queries (SelNet)

요약

본 논문은 Jiayi Sun, Guoliang Li, Nan Tang가 2021년 SIGMOD에 발표한 SelNet으로, 유사성 쿼리의 카디널리티 추정에 머신러닝을 적용한 최초의 주요 작업이다. 이 논문은 Exqutor의 직접적인 비교 대상이며, 유사성 검색 시스템에서 정확한 카디널리티 추정을 위해 신경망을 어떻게 사용할 수 있는지 보여준다.

SelNet의 핵심 아이디어는 단순하면서도 강력하다: 유사성 쿼리의 선택도(selectivity)를 예측하는 문제를 회귀 문제(regression problem)로 바라보고, 신경망을 사용하여 이를 학습하는 것이다. 선택도는 쿼리 결과의 개수를 데이터셋 전체 크기로 나눈 비율이다. 예를 들어, 100만 개의 벡터 중에서 1000개가 검색 조건을 만족한다면, 선택도는 0.001이다.

SelNet은 다음과 같은 특징을 갖는다. 첫째, 유사성 쿼리의 특성(벡터 차원, 거리 임계값, 쿼리 벡터의 특성 등)을 입력으로 받아, 선택도를 직접 예측한다. 둘째, 신경망 기반의 접근은 벡터 공간의 복잡한 특성을 자동으로 학습할 수 있다. 셋째, 다양한 데이터셋과 쿼리 패턴에 대해 충분한 학습 데이터가 있다면, 높은 정확도를 달성할 수 있다.

SelNet의 성능은 매우 인상적이다. 전통적인 통계 기반 추정 방법과 비교했을 때, 추정 오류를 평균 80% 이상 감소시킨다. 즉, 전통적 방법의 추정 오류가 5배라면, SelNet은 1배 이내의 오류로 줄일 수 있다는 뜻이다.

하지만 SelNet도 한계가 있다. 학습 데이터에 포함되지 않은 새로운 패턴의 쿼리에 대해서는 성능이 저하될 수 있다 (분포 이동, distribution shift). 또한 신경망의 구조와 하이퍼파라미터 선택에 따라 성능이 크게 달라질 수 있다. Exqutor는 이러한 SelNet의 한계를 개선하는 것을 목표로 한다.

상세분석

51.1 유사성 쿼리의 특성과 카디널리티 추정의 어려움

유사성 쿼리는 전통적인 조건부 쿼리(예: WHERE age > 30)와는 다른 특성을 가지고 있다. 전통적인 쿼리의 카디널리티는 주로 선택 조건(selection predicate)에 의존하고, 이는 상대적으로 예측하기 쉽다. 예를 들어, 나이가 균등하게 분포되어 있다면, "age > 30"을 만족하는 행의 비율은 거의 항상 일정하다.

반면 유사성 쿼리의 카디널리티는 여러 복잡한 요인에 의존한다:

첫째, 거리 임계값(distance threshold)이다. "벡터 q와의 거리가 d 이내인 벡터를 찾아라"라는 쿼리에서, d가 변하면 결과의 개수가 지수적으로 변할 수 있다. 벡터 공간에서는 거리가 증가함에 따라 포함되는 벡터의 개수가 변경의 차원 거듭제곱에 비례하므로 (d의 차원 거듭제곱), 매우 예민한 변화를 보인다.

둘째, 벡터 공간의 차원이다. 고차원 벡터 공간에서는 curse of dimensionality의 영향으로, 모든 벡터들이 서로 거의 비슷한 거리에 분포한다. 따라서 동일한 거리 임계값이라도, 차원에 따라 결과 개수가 크게 달라진다.

셋째, 데이터의 분포이다. 벡터가 균등하게 분포되어 있는지, 아니면 특정 영역에 밀집되어 있는지에 따라 결과 개수가 달라진다. 예를 들어, 이미지 데이터의 경우 특정 물체나 장면이 데이터셋에서 과다 대표될 수 있고, 이는 유사성 쿼리의 결과 개수에 영향을 미친다.

넷째, 쿼리 벡터의 특성이다. 쿼리 벡터가 데이터셋의 중심에 있는지, 아니면 주변부에 있는지에 따라 결과 개수가 달라진다. 또한 쿼리 벡터의 "인기도"(어떤 물체나 개념을 나타내는 정도)도 영향을 미칠 수 있다.

이러한 복잡한 요인들을 전통적인 통계 기반 방법으로 모두 고려하기는 매우 어렵다. SelNet은 신경망을 사용하여 이러한 요인들 간의 복잡한 관계를 자동으로 학습한다.

51.2 SelNet의 아키텍처와 특징 표현

SelNet의 신경망 아키텍처는 상대적으로 간단하다. 입력 계층, 여러 숨은 계층, 그리고 단일 출력 계층으로 구성된다. 출력은 선택도(selectivity) 값이다.

입력 특징(input features)은 유사성 쿼리를 나타낼 수 있는 모든 정보를 포함한다: - 벡터 차원(dimensionality): 벡터 공간의 차원 수 - 거리 임계값(distance threshold): 검색 반경 - 데이터셋 크기(dataset size): 전체 벡터의 개수 - 거리 통계(distance statistics): 평균 거리, 거리의 분산 등 - 벡터 분포 통계: 벡터들이 얼마나 밀집되어 있는지 등

이러한 특징들을 입력으로 사용하면, 신경망은 특징들 간의 복잡한 상호작용을 학습할 수 있다. 예를 들어, 차원이 높을수록 동일한 거리 임계값에서 더 많은 벡터가 포함되는 현상을 신경망이 학습할 수 있다.

특징 표현의 정규화(normalization)도 중요하다. 거리 임계값은 0부터 매우 큰 값까지 다양하므로, 신경망의 수렴을 위해 정규화가 필수적이다. SelNet은 특징들을 min-max 정규화 또는 z-score 정규화를 사용하여 0~1 범위 또는 평균 0, 분산 1로 변환한다.

51.3 학습 데이터 수집과 라벨링

SelNet을 학습하기 위해서는 대량의 (쿼리 특성, 선택도) 쌍의 학습 데이터가 필요하다. 논문에서는 다음과 같은 방법으로 학습 데이터를 수집한다.

첫째, 다양한 데이터셋을 준비한다. 논문에서는 실제 벡터 데이터셋(예: SIFT, GIST 특징) 뿐만 아니라, 합성 데이터(synthetic data)도 사용한다. 합성 데이터는 다양한 분포(균등 분포, 가우시안 분포, 클러스터링된 분포 등)를 가진 벡터들로 구성된다.

둘째, 각 데이터셋에 대해 다양한 쿼리를 생성한다. 거리 임계값을 체계적으로 변화시키면서 (예: 0.01, 0.05, 0.1, 0.2 등), 각 쿼리에 대한 정확한 선택도를 계산한다. 정확한 선택도는 각 쿼리를 실제로 전체 데이터셋에 대해 실행

하여 결과 개수를 센다.

셋째, 학습 데이터를 정규화하고 전처리한다. 선택도 값의 분포가 매우 비대칭적(skewed)일 수 있으므로, 로그 변환(log transformation)을 적용할 수 있다. 이는 신경망의 학습을 더 안정적으로 만든다.

학습 데이터의 크기는 신경망의 성능에 직접 영향을 미친다. 충분한 학습 데이터가 있으면 신경망이 복잡한 패턴을 학습할 수 있지만, 데이터가 부족하면 과적합(overfitting)의 위험이 있다.

51.4 신경망의 학습과 최적화

SelNet의 신경망 학습은 표준적인 지도 학습(supervised learning) 과정을 따른다. 손실 함수(loss function)로는 평균 제곱 오류(Mean Squared Error, MSE) 또는 평균 절대 오류(Mean Absolute Error, MAE)를 사용한다.

MSE는 큰 오류에 더 패널티를 주므로, 매우 잘못된 추정을 피하는 데 효과적이다. MAE는 오류의 크기에 직선적으로 패널티를 주므로, 이상치(outlier)에 덜 민감하다. 카디널리티 추정의 맥락에서는 MAE가 더 적절할 수 있다 (예: 실제 1000개일 때 99개와 1099개의 오류 크기는 같지만, 실무적 영향은 다를 수 있음).

최적화 알고리즘으로는 일반적으로 Adam 또는 SGD(Stochastic Gradient Descent)를 사용한다. Adam은 빠른 수렴을 제공하고 하이퍼파라미터에 덜 민감하므로, 많은 경우 기본 선택이 된다.

정규화(regularization)도 중요하다. L1 또는 L2 정규화를 사용하여 가중치의 크기를 제한하면, 과적합을 방지할 수 있다. 조기 종료(early stopping)도 효과적인데, 검증 데이터셋에서의 성능이 더 이상 개선되지 않으면 학습을 멈춘다.

51.5 평가 지표와 성능 분석

SelNet의 성능을 평가하기 위해 여러 지표를 사용한다:

첫째, 추정 오류(estimation error)이다. 일반적으로 q-error (quantile error)를 사용한다. 이는 예측값과 실제값의 최대 비율을 나타낸다. 예를 들어, 실제값이 1000이고 예측값이 500이면 q-error는 2이다 ($\max(1000/500, 500/1000) = 2$). q-error는 상대적 오류를 나타내므로, 절대적인 값의 크기에 관계없이 오류의 심각성을 평가할 수 있다.

둘째, 95 percentile q-error이다. 모든 쿼리에 대한 q-error의 95 백분위수 값으로, 매우 나쁜 추정이 얼마나 자주 발생하는지를 나타낸다.

셋째, Mean Absolute Percentage Error (MAPE)이다. 이는 평균적인 백분율 오류를 나타낸다.

SelNet의 실험 결과에 따르면: - 전통적인 통계 기반 추정 대비 평균 q-error를 3~5배에서 1.5~2배로 개선 - 일부 특정 벡터 차원과 거리 범위에서는 거의 완벽한 추정 달성 - 학습 데이터가 충분한 분포 범위에서는 $q\text{-error} < 2$ 달성

51.6 SelNet의 한계와 실무적 과제

SelNet은 강력한 방법이지만, 실무 적용에 있어 여러 한계가 있다.

첫째, 분포 이동(distribution shift)이다. 모델이 학습한 데이터의 분포와 실제 운영 환경의 데이터 분포가 다르면, 성능이 저하된다. 예를 들어, 학습 데이터는 거리 임계값이 0.1~0.5 범위의 쿼리만 포함했는데, 실제로는 0.05나 1.0 범위의 쿼리가 들어올 수 있다.

둘째, 데이터 변화이다. 시간이 지남에 따라 데이터셋의 분포가 변할 수 있다. 예를 들어, 추천 시스템의 벡터가 지속적으로 업데이트되면, 이전의 거리 분포와는 다른 패턴이 나타날 수 있다.

셋째, 신경망의 해석가능성이 낮다. 신경망이 어떤 특징을 중요하게 여기는지, 왜 특정 쿼리에서 오류가 발생하는지 이해하기 어렵다. 이는 모델의 신뢰도를 낮출 수 있다.

넷째, 새로운 인덱스 구조에의 적응이다. 만약 IVF 인덱싱을 사용하다가 HNSW로 변경하면, 모델의 성능이 저하될 수 있다. 인덱스 구조도 입력 특징에 포함되어야 하며, 각 인덱스 구조에 대해 별도의 학습이 필요할 수 있다.

51.7 Exqutor와의 차이점 및 개선점

Exqutor는 SelNet의 개념을 바탕으로 하지만, 여러 가지 측면에서 개선을 시도한다:

첫째, 더 정교한 특징 엔지니어링이다. Exqutor는 벡터 공간의 고유한 특성을 더 잘 포착할 수 있는 특징들을 설계한다. 예를 들어, 거리 분포의 고전적인 통계량 외에도, 벡터 공간의 국소적 밀도(local density)나 곡률(curvature) 같은 기하학적 특성을 고려할 수 있다.

둘째, 불확실성 정량화(uncertainty quantification)이다. Exqutor는 단순히 점 추정(point estimate)만 제공하는 것이 아니라, 추정값에 대한 신뢰도(confidence interval) 또는 불확실성(uncertainty)도 제공할 수 있다. 이는 쿼리 최적화에서 위험도를 고려한 의사결정을 가능하게 한다.

셋째, 적응형 학습(adaptive learning)이다. Exqutor는 새로운 쿼리 패턴이 발생할 때, 모델을 점진적으로 업데이트할 수 있다. 이를 통해 분포 이동에 대한 적응성을 높일 수 있다.

넷째, 해석가능성 개선이다. Exqutor는 트리 기반 모델(예: XGBoost, Random Forest) 또는 선형 모델을 사용할 수 있으며, 이들은 신경망보다 해석가능성이 높다.

다섯째, 실행 비용의 고려이다. Exqutor는 부분 실행을 통한 실제 카디널리티 확인과 머신러닝 기반 추정을 결합하여, 각 쿼리의 특성과 실행 예산(execution budget)에 따라 최적의 방식을 선택할 수 있다.

51.8 본 논문과의 관계

SelNet은 유사성 쿼리의 카디널리티 추정에 머신러닝을 처음 적용한 중요한 작업이다. 이는 기존의 통계 기반 추정 방법이 유사성 쿼리에 부적합하다는 것을 보여주었고, 머신러닝이 이 문제를 해결할 수 있는 강력한 도구임을 증명했다.

Exqutor는 SelNet의 핵심 아이디어를 계승하면서도, 다음과 같은 측면에서 개선을 이룬다: - 더 강력하고 정확한 학습 모델 - 실행 비용 제약을 고려한 최적화 - 부분 실행과 학습 기반 추정의 효과적인 결합 - 더 포괄적인 실험과 평가

추가 제기 문제

1. **모델의 일반화**: SelNet 모델이 학습한 데이터와 다른 분포의 새로운 데이터에 얼마나 잘 일반화되는가? 특히 매우 높은 차원이나 매우 낮은 차원의 데이터에는 어떻게 적응하는가?
2. **온라인 학습과 적응**: 실제 운영 환경에서 지속적으로 들어오는 새로운 쿼리들을 어떻게 활용하여 모델을 개선할 것인가? 모델 재학습의 빈도와 비용은?
3. **인덱스 구조의 영향**: 동일한 데이터에 대해 다양한 인덱스 구조(IVF, HNSW, LSH 등)를 사용할 때, 모델의 성능이 어떻게 달라지는가? 인덱스 구조별로 별도의 모델을 학습해야 하는가?
4. **카디널리티 범위**: 매우 작은 카디널리티(< 10)나 매우 큰 카디널리티($> \text{백만}$)의 경우, 모델의 성능이 충분한가? 이러한 극단적인 경우를 특별히 처리해야 하는가?
5. **해석가능성**: 신경망이 특정 쿼리에서 큰 오류를 범한 이유를 사용자나 관리자가 이해할 수 있는 방법이 있는가?